Lecture 4
Top-Down Explicit Search

Armando Solar-Lezama
with content from Jack Feser

Quick Intro to Functional Idioms

```
map f lst = case lst of
                []          -> []
                head:rest -> f(head) : (map f rest)
```

- Applies *f* to every element in the list

```
filter p lst = case lst of
                 []  -> []
                 head:rest -> if p(head) then head: (filter p rest)
                                         else (filter p rest)
```

- Removes any element x for which p(x) is false

foldl definition

```
foldl binop start lst = case lst of
                          []  -> start
                           head:rest -> (foldl binop (binop start head) rest)
```

- Apply the binary operation binop from left to right to the list

foldl definition

```
foldl binop start lst = case lst of
                          []  -> start
                          head:rest -> (foldl binop (binop start head) rest)
```

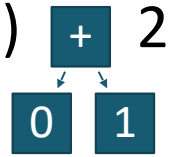
- Apply the binary operation binop from left to right to the list

foldl (+) 0 1:2:3:4:[]

foldl definition

```
foldl binop start lst = case lst of
                          []  -> start
                           head:rest -> (foldl binop (binop start head) rest)
```

- Apply the binary operation binop from left to right to the list

foldl (+) 

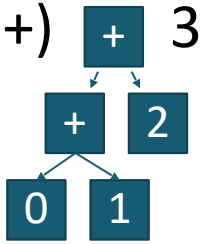
2:3:4:[]

foldl definition

```
foldl binop start lst = case lst of
                          []  -> start
                           head:rest -> (foldl binop (binop start head) rest)
```

- Apply the binary operation binop from left to right to the list

foldl (+) + 3:4:[]

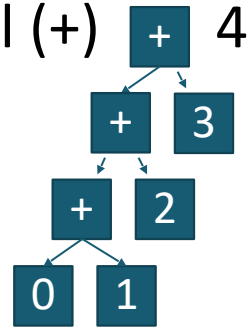


foldl definition

```
foldl binop start lst = case lst of
                          []  -> start
                           head:rest -> (foldl binop (binop start head) rest)
```

- Apply the binary operation binop from left to right to the list

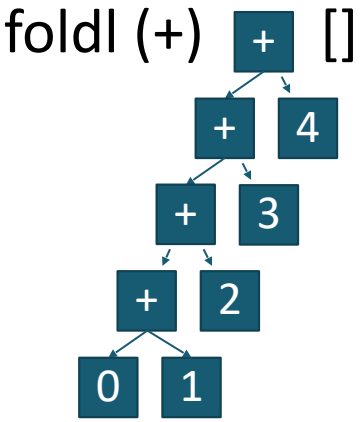
foldl (+) + 4:[]



foldl definition

```
foldl binop start lst = case lst of
                          []  -> start
                           head:rest -> (foldl binop (binop start head) rest)
```

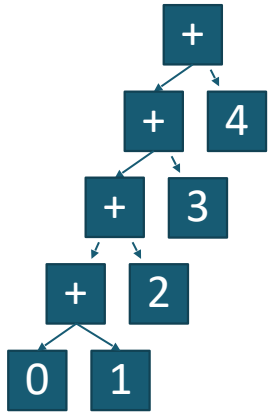
- Apply the binary operation binop from left to right to the list



foldl definition

```
foldl binop start lst = case lst of
                          []  -> start
                           head:rest -> (foldl binop (binop start head) rest)
```

- Apply the binary operation binop from left to right to the list



Example

- Teacher has a gradebook
- Wants to drop the lowest score for each student
- Needs a program to do this operation

	Midterm 1	Midterm 2	Final
Student A	~~71~~	75	83
Student B	90	~~87~~	95
Student C	~~68~~	77	80

Example

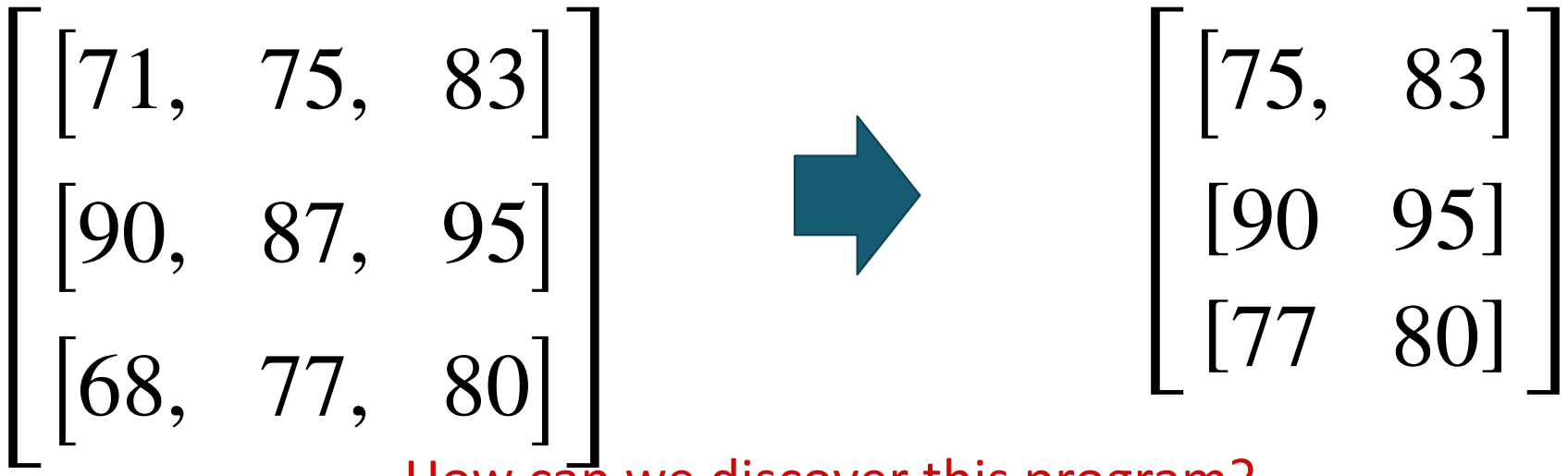
Drop the smallest element from each list

$$\begin{bmatrix} [71, & 75, & 83] \\ [90, & 87, & 95] \\ [68, & 77, & 80] \end{bmatrix}$$



$$\begin{bmatrix} [75, & 83] \\ [90 & 95] \\ [77 & 80] \end{bmatrix}$$

Example



How can we discover this program?

```
dropmins x = map dropmin x
    where dropmin y = filter isNotMin y
        where isNotMin z = foldl h False y
            where h t w = t || (w < z)
```

Defining the language

expr = var

| $\lambda x\ .\ \ expr$

| **filter** *expr expr*

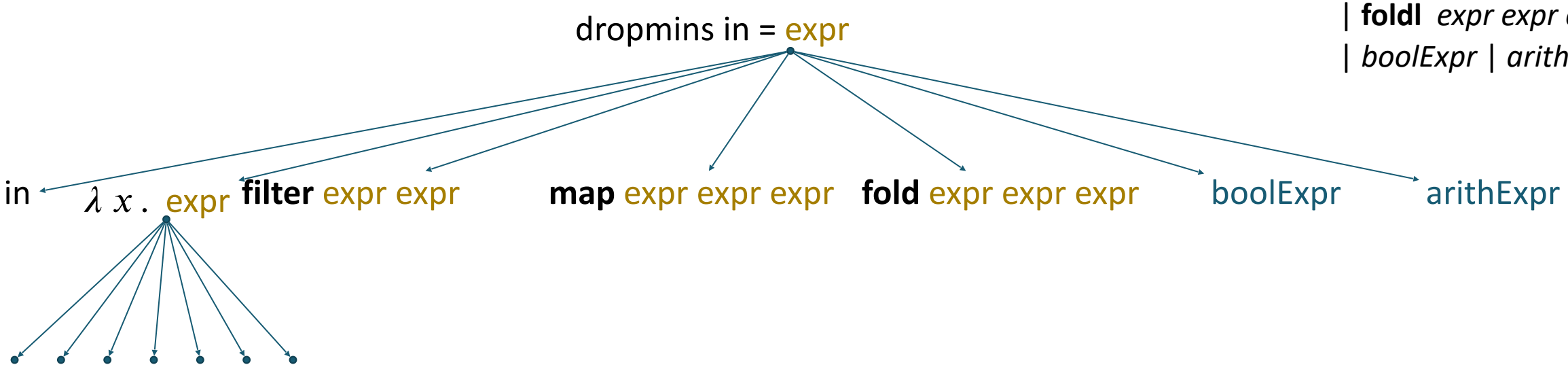
| **map** *expr expr*

| **foldl** *expr expr expr*

| *boolExpr* | *arithExpr*

Top-down search

expr = var
| $\lambda x\ .\ expr$
| **filter** *expr expr*
| **map** *expr expr*
| **foldl** *expr expr expr*
| *boolExpr* | *arithExpr*

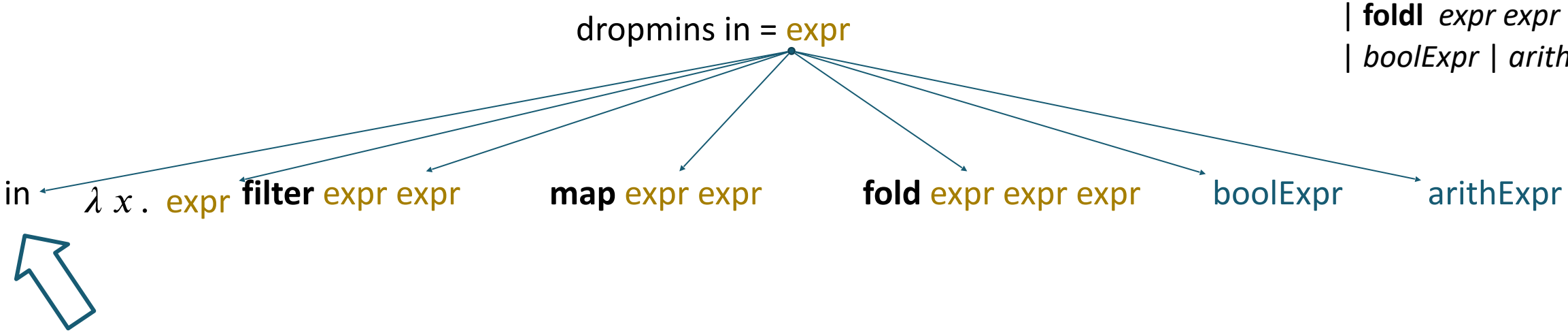


Many of these programs can be eliminated before having to complete them!

How?

Top-down search

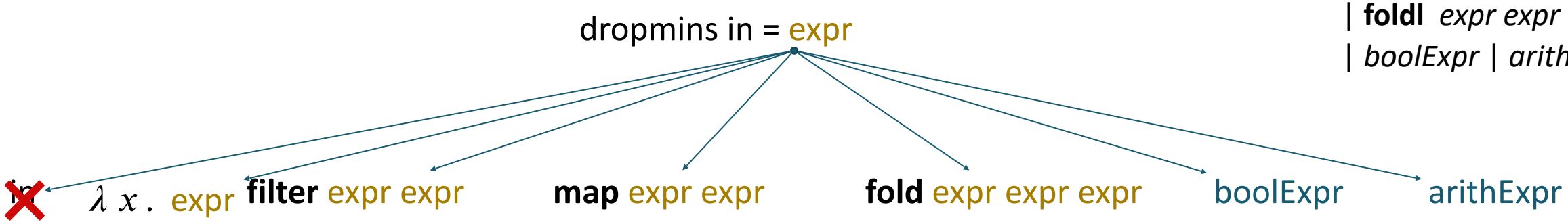
expr = var
| $\lambda x\ .\ expr$
| **filter** *expr expr*
| **map** *expr expr*
| **foldl** *expr expr expr*
| *boolExpr* | *arithExpr*



This is a fully concrete program, and
it clearly doesn't match the examples

Top-down search

expr = var
| $\lambda x\ .\ expr$
| **filter** *expr expr*
| **map** *expr expr*
| **foldl** *expr expr expr*
| *boolExpr* | *arithExpr*



This program has a missing expression, but we can already tell it will not work. Why not?

Types

Our simple language supports an infinite set of types of 3 basic kinds

$$\tau \quad := \quad Int \quad | \quad Bool \quad | \quad [\tau] \quad | \quad \tau \rightarrow \tau$$

Integer

Boolean

List of some type

Function from some type to some other type

Types

$$\left[\begin{array}{l}[71,\ 75,\ 83] \\ [90,\ 87,\ 95] \\ [68,\ 77,\ 80]\end{array}\right] \qquad \left[\begin{array}{l}[75,\ 83] \\ [90\ \ 95] \\ [77\ \ 80]\end{array}\right]$$

$$[\ [Int]\ ] \qquad [\ [Int]\ ]$$

Input and output types are lists of lists of integers

Types

Each element in our language has a type given by a *typing rule*

$$\frac{premises}{C \vdash expr : \tau}$$

A typing rule like the one above states that $expr$ has type τ in a context C as long as all the premises are satisfied

- A context simply tracks information about the type of any variables

Types

Each element in our language has a type given by a *typing rule*

$$\frac{C \text{ says var} \quad has\ type\ \tau}{C \vdash \text{var} : \tau}$$

$$\frac{f : \tau_1 \to \tau_2 \qquad epxr : \tau_1}{C \vdash f\ expr : \tau_2}$$

$$\frac{C, x : \tau_1 \vdash expr : \tau_2}{C \vdash \lambda\ \text{x}\ .\ \text{expr} : \tau_1 \to \tau_2}$$

$$\frac{}{map : (\tau_1 \to \tau_2) \to [\tau_1] \to [\tau_2]}$$

$$\frac{}{foldl : (\tau_{start} \to \tau_{lst} \to \tau_{start}) \to \tau_{start} \to [\tau_{lst}] \to \tau_{start}}$$

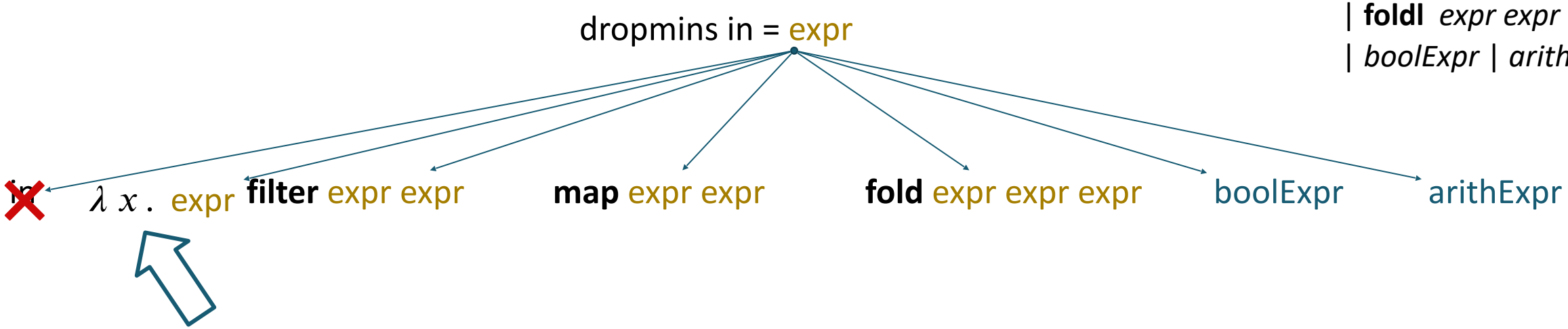
$$\frac{}{boolExpr : Bool}$$

$$\frac{}{filter : (\tau \to Bool) \to [\tau] \to [\tau]}$$

$$\frac{}{intExpr : Int}$$

Type-based pruning

expr = var
| $\lambda x\ .\ expr$
| **filter** *expr expr*
| **map** *expr expr*
| **foldl** *expr expr expr*
| *boolExpr* | *arithExpr*



$$\frac{expr : \tau_2 \ assuming \ x : \tau_1}{\lambda \text{ x . expr} : \tau_1 \rightarrow \tau_2}$$

Based on the rule, this expression will have a type $\tau_1 \rightarrow \tau_2$

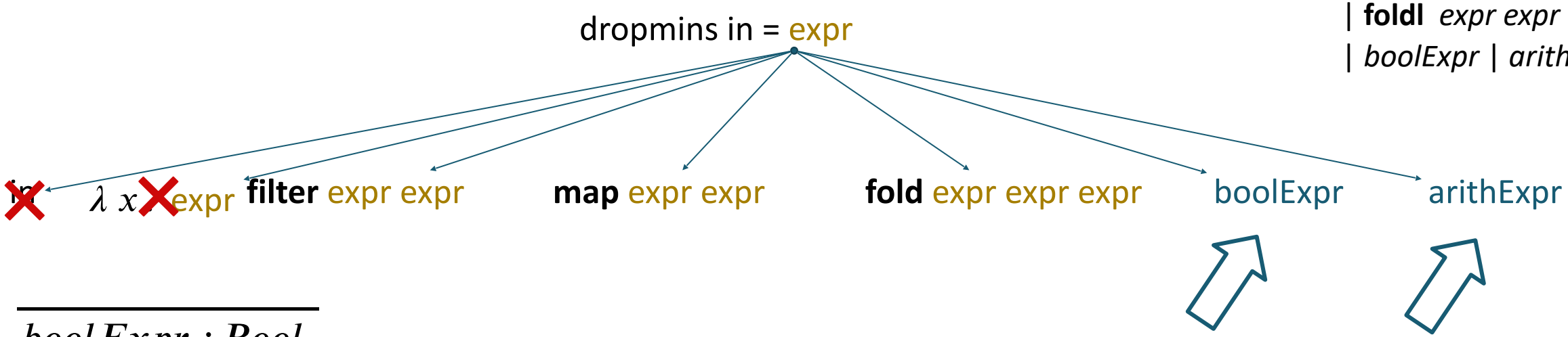
But we know the output must have type [[*Int*]]

There is no way those types can be made equal,

so we can discard this expression!

Type-based pruning

expr = var
| $\lambda x\ .\ expr$
| **filter** *expr expr*
| **map** *expr expr*
| **foldl** *expr expr expr*
| *boolExpr* | *arithExpr*



$$\frac{}{boolExpr : Bool}$$

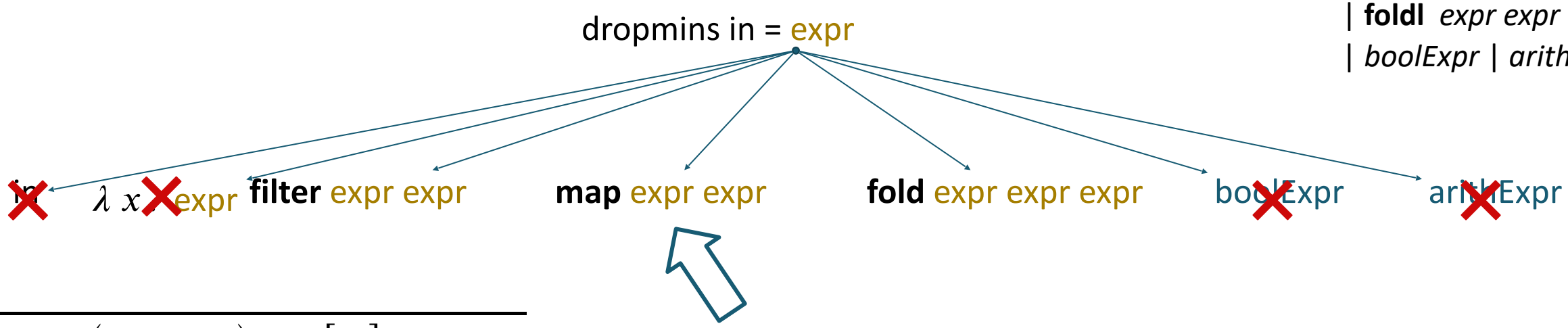
$$\frac{}{intExpr : Int}$$

With the same reasoning we can discard both of these expressions

They cannot possibly have the correct type

Type-based pruning

expr = var
| $\lambda x\ .\ expr$
| **filter** *expr expr*
| **map** *expr expr*
| **foldl** *expr expr expr*
| *boolExpr* | *arithExpr*



$$map : (\tau_1 \to \tau_2) \to [\tau_1] \to [\tau_2]$$

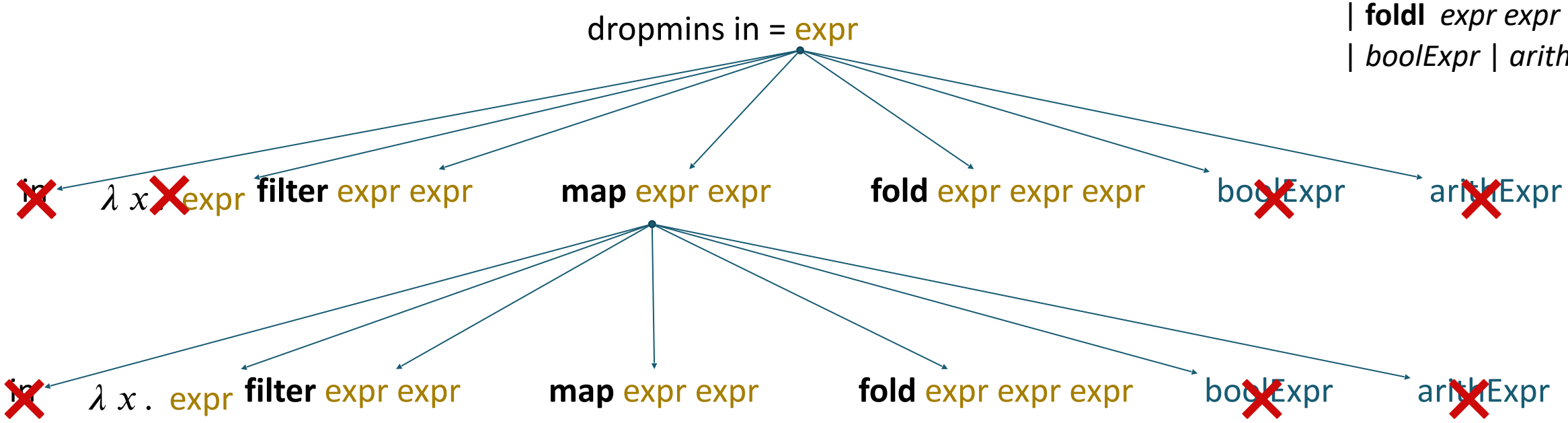
We know the output should be $[\ [Int]\ ]$

This means the first expr must be $\tau_1 \to [Int]$

otherwise the types won't match

Type-based pruning

expr = var
| $\lambda x\ .\ expr$
| **filter** *expr expr*
| **map** *expr expr*
| **foldl** *expr expr expr*
| *boolExpr* | *arithExpr*



We can quickly dismiss many possible expressions because they cannot produce the type $\tau_1 \rightarrow [Int]$

We can do better

Key observation: in some cases we can propagate examples to the sub-components

Example

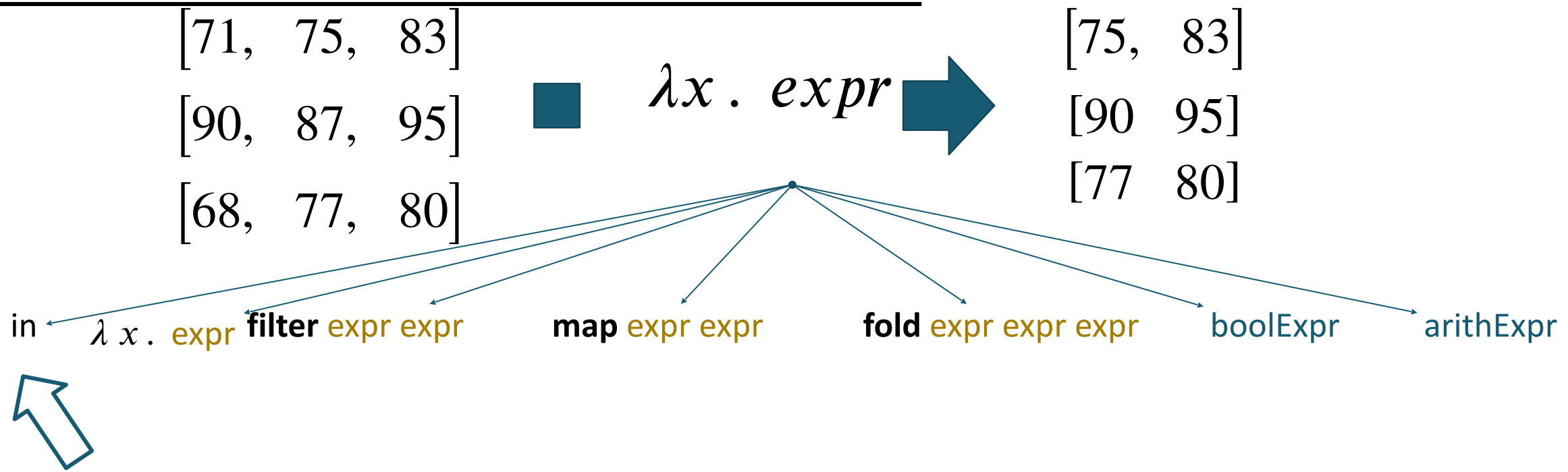
$$\begin{bmatrix} [71, & 75, & 83] \\ [90, & 87, & 95] \\ [68, & 77, & 80] \end{bmatrix} \quad \text{map } in\ \lambda x\ .\ expr \quad \begin{bmatrix} [75, & 83] \\ [90 & 95] \\ [77 & 80] \end{bmatrix}$$

$$[71, \quad 75, \quad 83] \qquad [75, \quad 83]$$

$$[90, \quad 87, \quad 95] \quad \lambda x\ .\ expr \quad [90 \quad 95]$$

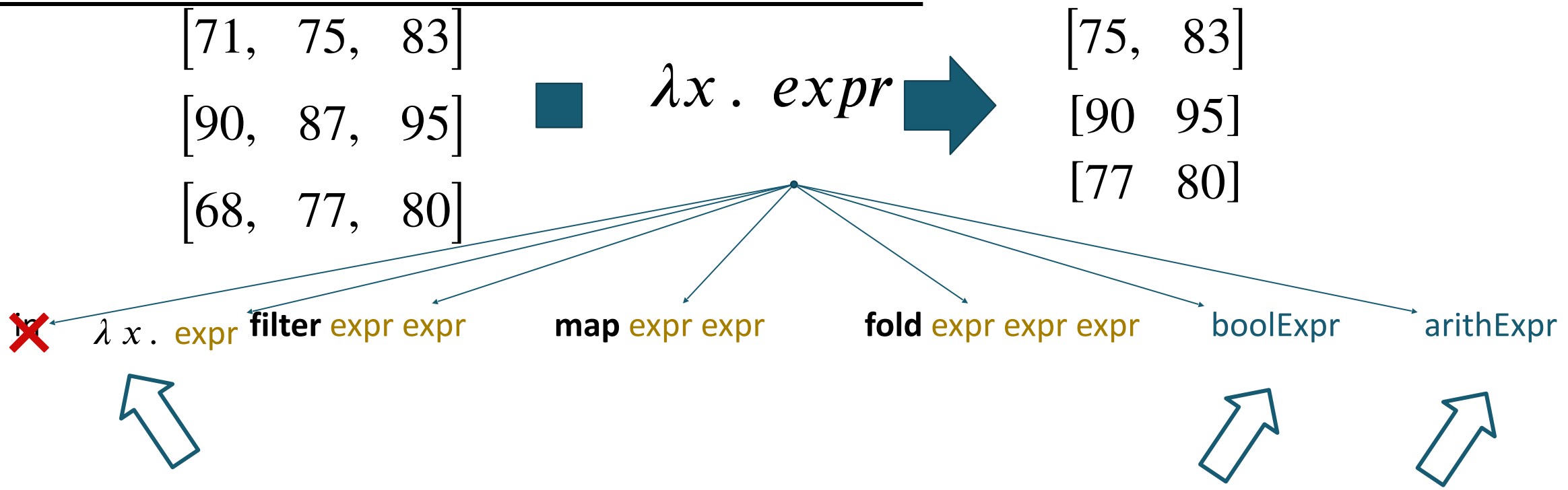
$$[68, \quad 77, \quad 80] \qquad [77 \quad 80]$$

Continuing the search



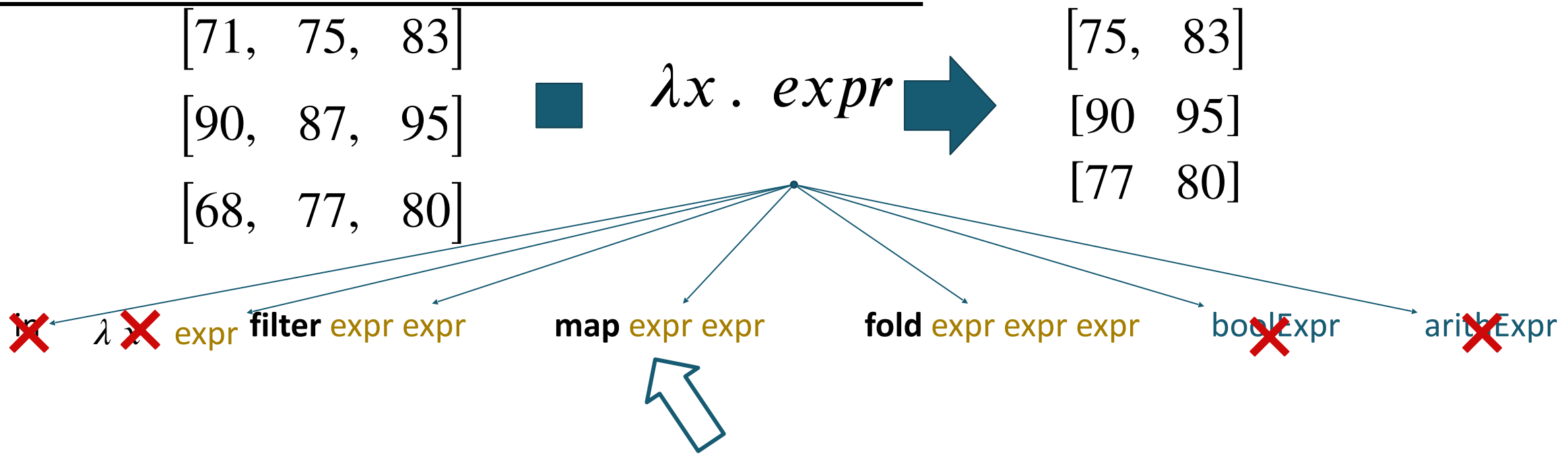
This is a fully concrete program, and
it clearly doesn't match the examples

Continuing the search



Again, these are all of the wrong type

Continuing the search



We have a rule to propagate examples to

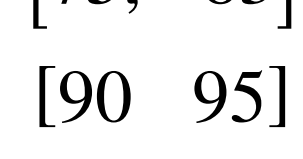
map x $\lambda y\ .\ \ expr$

But it clearly cannot work in this case because the input and output lists are of different sizes

Works for filter as well

$$\begin{bmatrix} 71, & 75, & 83 \end{bmatrix}$$
$$\begin{bmatrix} 90, & 87, & 95 \end{bmatrix}$$
$$\begin{bmatrix} 68, & 77, & 80 \end{bmatrix}$$

- λx . filter (λy. expr) x



$$\begin{bmatrix} 75, & 83 \end{bmatrix}$$
$$\begin{bmatrix} 90 & 95 \end{bmatrix}$$
$$\begin{bmatrix} 77 & 80 \end{bmatrix}$$

x= [71, 75, 83] , y=71
x= [71, 75, 83] , y=75
x= [71, 75, 83] , y=83
x= [90, 87, 95] , y=90
x= [90, 87, 95] , y=87
x= [90, 87, 95] , y=95
x= [68, 77, 80] , y=68
x= [68, 77, 80] , y=77
x= [68, 77, 80] , y=80

- expr



false
true
true
true
false
true
false
true
true

Deduction rules

All of these tricks are codified into deduction rules

Two kinds of rules

- Rules that tell you that a candidate is not going to work
  - for example if input list has different length from output list, map f in will not work
- Rules that tell you how to propagate input/outputs to new expressions

Rule format: when it won't work

$$\frac{conditions}{Examples,\ expression[f] \vdash f : \bot}$$



Expression involving a function f

Example: rule for map

$$\mathcal{E} = \bigcup_{1 \le i \le n} \{[a_{i1}, \ldots, a_{ig(i)}] \mapsto [b_{i1}, \ldots, b_{ih(i)}]\}$$

$$\frac{\exists i, j, k, m. \left(\begin{array}{c} 1 \le i \le n \wedge 1 \le j \le n \wedge 1 \le k \le g(i) \wedge \\ 1 \le m \le h(j) \wedge a_{ik} = a_{jm} \wedge b_{ik} \neq b_{jm} \end{array} \right)}{\mathcal{E}, \lambda x.\ \texttt{map}\ f\ x \vdash f : \bot}$$

If the same input value is mapped to multiple output values, then there is no possible function *f*.

Rule format: when we can propagate

$$\frac{conditions}{Examples,\ expression[f] \vdash f : New\ Examples}$$

Example rules for map

$$\frac{\begin{array}{c}\mathcal{E} = \bigcup_{1 \le i \le n} \{[a_{i1}, \ldots, a_{ig(i)}] \mapsto [b_{i1}, \ldots, b_{ig(i)}]\} \\ \mathcal{E}' = \bigcup_{1 \le i \le n} \{a_{i1} \mapsto b_{i1}, \ldots a_{ig(i)} \mapsto b_{ig(i)}\} \\ ((a,b) \in \mathcal{E}' \wedge (a',b') \in \mathcal{E}' \wedge a = a') \Rightarrow b = b'\end{array}}{\mathcal{E}, \lambda x.\, \texttt{map}\ f\ x \vdash f : \mathcal{E}'}$$